

TEXT NORMALIZATION 2: WORD NORMALIZATION & OTHERS

Natural Language Processing

Word Normalization

- Word (vocabulary) normalization is the task of putting words or tokens in a standard format.
- Tokens/words that mean similar things are combined into a single, normalized form.
 - U.S.A. or USA
 - uhhuh or uh-huh
 - Fed or fed
 - am, is, be, are
- Reduces vocabulary size and improves text processing.

Types of Word Normalization

- Case folding
- Synonym substitution
- Stemming
- Lemmatization

Case Folding

- **Case folding** is the simplest form of word normalization - mapping everything to lowercase.
- It is very helpful for generalization in many tasks, such as information retrieval or speech recognition.
- For sentiment analysis and other text classification tasks, information extraction, and machine translation, by contrast, the case can be quite helpful, and case folding is generally not done.
- Therefore, we apply case folding when the upper/lowercase letters do not have any effect on the further text processing.

Case Folding

```
tokens = ['House', 'Visitor', 'Center']  
normalized_tokens = [x.lower() for x in tokens]  
print(normalized_tokens)
```

Case Folding

```
tokens = ['House', 'Visitor', 'Center']  
normalized_tokens = [x.lower() for x in tokens]  
print(normalized_tokens)
```

Output

```
['house', 'visitor', 'center']
```

Synonym Substitution

- **Synonym substitution** is a technique in which we replace a word with one of its synonyms.
- There are cases where it is useful to replace words with their synonyms or abbreviations with their full forms.

“We are studying NLP.” => “We are reading NLP.”

“I’ll be there at 2pm” => “I will be there at 2 pm”

Synonym Substitution

- It makes text data more readable and understandable.
- It can improve the performance of NLP algorithms that rely on word or phrase-level analysis.
- It can lead to the loss of information, as the meaning of a sentence or text can change based on the context of words.
- There are scenarios in which synonym substitution of some sort may make sense, such as:
 - Search engines
 - Data augmentation
 - Scoring the robustness of your NLP
 - Adversarial NLP

Synonym Substitution

```
text = ["I'll be there at 2pm.", "We are studying NLP."]
for indx, t in enumerate(text):
    print("Original Text", indx+1, ": ", t)

synonyms = {"I'll": "I will", "2pm": "2 pm", "studying": "reading"}
for indx, t in enumerate(text):
    for key, value in synonyms.items():
        t = t.replace(key, value)
    text[indx] = t

for indx, t in enumerate(text):
    print("Normalized Text", indx+1, ": ", t)
```

Synonym Substitution

```
text = ["I'll be there at 2pm.", "We are studying NLP."]
for indx, t in enumerate(text):
    print("Original Text", indx+1, ": ", t)

synonyms = {"I'll": "I will", "2pm": "2 pm", "studying": "reading"}
for indx, t in enumerate(text):
    for key, value in synonyms.items():
        t = t.replace(key, value)
    text[indx] = t

for indx, t in enumerate(text):
    print("Normalized Text", indx+1, ": ", t)
```

Output

```
Original Text 1 : I'll be there at 2pm.
Original Text 2 : We are studying NLP.
Normalized Text 1 : I will be there at 2 pm.
Normalized Text 2 : We are reading NLP.
```

Synonym Substitution

Try WordNet synonyms (nltk)

```
text = ["I'll be there at 2pm.", "We are studying NLP."]
for indx, t in enumerate(text):
    print("Original Text", indx+1, ": ", t)

synonyms = {"I'll": "I will", "2pm": "2 pm", "studying": "reading"}
for indx, t in enumerate(text):
    for key, value in synonyms.items():
        t = t.replace(key, value)
    text[indx] = t

for indx, t in enumerate(text):
    print("Normalized Text", indx+1, ": ", t)
```

Output

```
Original Text 1 : I'll be there at 2pm.
Original Text 2 : We are studying NLP.
Normalized Text 1 : I will be there at 2 pm.
Normalized Text 2 : We are reading NLP.
```

Stemming

- **Stemming** is a process that removes the inflectional part of words (prefixes and suffixes) by stripping them off and transforming them into their root or base form called **stem**.
- For example, stemming would reduce the words “running,” “runner,” and “runs” to their stem “run.”
- A stem is not required to be a properly spelled word but merely a token or label representing several possible spellings of a word.
- **Stems** — the central morpheme of the word, supplying the main meaning.
- **Affixes** — adding “additional” meanings of various kinds. (prefixes and suffixes)

Stemming

- Stemming helps improve the performance of information retrieval, text classification, and other NLP tasks.
- It does not consider the semantic meaning of the words while reducing them. The root word formed may have lost its semantic meaning.
- It is less commonly used in modern systems since it commits errors of both over-generalizing (mapping policy to police) and under-generalizing (not mapping European to Europe).

Stemming

```
import nltk
from nltk.stem import PorterStemmer
from nltk.tokenize import word_tokenize
nltk.download('punkt_tab')

sentence = "The quick brown foxes were jumping over the lazy dogs."
words = word_tokenize(sentence)

stemmer = PorterStemmer()
stemmed_words = [stemmer.stem(word) for word in words]

print("Original words:", words)
print("Stemmed words:", stemmed_words)
```

Stemming

```
import nltk
from nltk.stem import PorterStemmer
from nltk.tokenize import word_tokenize
nltk.download('punkt_tab')

sentence = "The quick brown foxes were jumping over the lazy dogs."
words = word_tokenize(sentence)

stemmer = PorterStemmer()
stemmed_words = [stemmer.stem(word) for word in words]

print("Original words:", words)
print("Stemmed words:", stemmed_words)
```

Output

```
Original words: ['The', 'quick', 'brown', 'foxes', 'were',
'jumping', 'over', 'the', 'lazy', 'dogs', '.']
Stemmed words: ['the', 'quick', 'brown', 'fox', 'were', 'jump',
'over', 'the', 'lazi', 'dog', '.']
```

Lemmatization

- **Lemmatization** is the task of determining that two words have the same root (**lemma**), despite their surface differences.
- The root or base word is known as **stem** during stemming while it is known as **lemma** during lemmatization.

am, are, is => **be**

car, cars, car's, cars' => **car**

*He is reading detective stories => He **be read** detective **story***

Lemmatization

- The most sophisticated methods for lemmatization involve **complete morphological parsing** of the word.
- **Morphology** is the study of the way words are built up from smaller meaning-bearing units called **morphemes**.
- Two broad classes of morphemes can be distinguished: **stems** and **affixes**.
- For example, the word “fox” consists of one morpheme (the morpheme “fox”) and the word “cats” consists of two: the morpheme “cat” and the morpheme “s”.
- A **morphological parser** takes a word like “cats” and parses it into the two morphemes “cat” and “s”.

Lemmatization

- Any NLP task that wants to “react” the same for multiple different spellings of the same basic root word can benefit from a lemmatizer.
- Lemmatization can make your model more general, but it can also make your model less precise because it will treat all spelling variations of a given root word the same.

chat, chatter, chatty, chatting => chat (chatter?)
bank, banked, banking => bank (river bank?)

possible mapping

Lemmatization

```
import spacy

nlp = spacy.load("en_core_web_sm")
doc = nlp("is banker better good goods best") # tokens
for token in doc:
    print(token.text, "=>", token.lemma_)
```

Lemmatization

```
import spacy

nlp = spacy.load("en_core_web_sm")
doc = nlp("is banker better good goods best") # tokens
for token in doc:
    print(token.text, "=>", token.lemma_)
```

Output

```
is => be
banker => banker
better => well
good => good
goods => good
best => well
```

Lemmatization

Try WordNetLemmatizer (nltk)

```
import spacy

nlp = spacy.load("en_core_web_sm")
doc = nlp("is banker better good goods best") # tokens
for token in doc:
    print(token.text, "=>", token.lemma_)
```

Output

```
is => be
banker => banker
better => well
good => good
goods => good
best => well
```

Punctuation Removal

- **Punctuation removal** is the process of removing special characters and punctuation marks from the text.
- This technique is useful when working with text data containing many punctuation marks, which can make the text harder to process.
- It removes unnecessary characters, making the text cleaner and easier to process. May improve the performance of NLP algorithms.
- It can lead to loss of information, as punctuation marks can indicate sentiment or emphasis.

Input: “ Good Morning! How are you? ”
Output: “Good Morning How are you”

Punctuation Removal

```
import string

text = " Good Morning! How are you? "
print(string.punctuation)
text_remove_mapping = text.maketrans("", "", string.punctuation)
text = text.translate(text_remove_mapping)
print(text.strip())
```

Punctuation Removal

```
import string

text = " Good Morning! How are you? "
print(string.punctuation)
text_remove_mapping = text.maketrans("", "", string.punctuation)
text = text.translate(text_remove_mapping)
print(text.strip())
```

Output

```
!"#$%&'()*+,-./:;<=>?@[\\]^_`{|}~
Good Morning How are you
```


Punctuation Removal

Options: re.sub()

```
import string

text = " Good Morning! How are you? "
print(string.punctuation)
text_remove_mapping = text.maketrans("", "", string.punctuation)
text = text.translate(text_remove_mapping)
print(text.strip())
```

Output

```
!"#$%&'()*+,-./:;<=>?@[\\]^_`{|}~
Good Morning How are you
```

Stop Words Removal

- **Stop words** are commonly used words that have very little meaning or do not have significant meaning on their own, such as “a,” “an,” “the,” or “in.”
- Stop words are typically excluded from NLP applications because mostly, they do not contribute much to the meaning or context of the text.
- In most text classification applications, using a stop word list doesn't improve performance, and so it is more common to make use of the entire vocabulary and not use a stop word list.
- Some common stopwords in English:
 - articles (a, an, the); conjunctions (and, but, or); prepositions (in, on, at); pronouns (he, she, it, they); auxiliary verbs (is, are, was, were)

Stop Words Removal

```
import nltk
from nltk.corpus import stopwords
nltk.download('stopwords')

text = "This is an example sentence with the stopwords."
stop_words = set(stopwords.words("english"))
words = text.split()
filtered_words = [word for word in words if word not in stop_words]
text = " ".join(filtered_words)
print(text)
```

Stop Words Removal

```
import nltk
from nltk.corpus import stopwords
nltk.download('stopwords')

text = "This is an example sentence with the stopwords."
stop_words = set(stopwords.words("english"))
words = text.split()
filtered_words = [word for word in words if word not in stop_words]
text = " ".join(filtered_words)
print(text)
```

Output

This example sentence stopwords.

Stop Words Removal

Options: spaCy, gensim, TextBlob

```
import nltk
from nltk.corpus import stopwords
nltk.download('stopwords')

text = "This is an example sentence with the stopwords."
stop_words = set(stopwords.words("english"))
words = text.split()
filtered_words = [word for word in words if word not in stop_words]
text = " ".join(filtered_words)
print(text)
```

Output

This example sentence stopwords.

References

- Daniel Jurafsky and James H. Martin. Speech and Language Processing, 3rd edition, 2024.
- Hobson Lane and Maria Dyschel. Natural Language Processing in Action, 2nd Edition, 2025.
- <https://www.scaler.com/topics/nlp/what-is-text-normalization-in-nlp/>
- https://maelfabien.github.io/machinelearning/NLP_8/
- <https://spotintelligence.com/2023/01/25/text-normalization-techniques-nlp/>
- <https://saturncloud.io/glossary/stemming/>
- <https://spotintelligence.com/2022/12/10/stop-words-removal/>